GUSTO POS

Complete Project Reference

Zero-Context Guide — Architecture, DB Design, API Layout & Data Flow

1. What Is This Project?

Gusto POS is a **multi-iteration restaurant Point-of-Sale ecosystem**. Customers scan a QR code, browse the menu, place orders and pay. Kitchen staff see orders in real time via a Kitchen Display System (KDS). Managers get analytics and inventory. The Desktop folder contains several independent versions at different maturity levels.

2. Top-Level Folder Map

```
C:\Users\Adithya\Desktop\
■
■■■ gusto_pos/              ← PRODUCTION BUILD  (FastAPI + Next.js + SQLite)
■■■ pos_1st_cut/            ← ADVANCED BUILD    (FastAPI Async + PostgreSQL + Redis + WebSocket)
■■■ restaurant-pos-api/     ← MINIMAL API       (Express + SQLite, no frontend)
■■■ restaurant-sim/         ← DEMO SIMULATOR    (Express + JSON file store)
■■■ demo1/                  ← THIS REPO — diagnostic workspace, fix branches
■   ■■■ gusto_pos_fix/      ← Fix iteration with Alembic migrations
■
■■■ Cricket_AI/             ← Experiment (Groq AI + audio)
■■■ socket/                 ← WebSocket experiments
■■■ tutorial/               ← Learning reference
■
■■■ App.tsx                 ← Root multi-tab POS app (Vite + React 19)
■■■ index.tsx               ← React entry point
■■■ types.ts                ← Shared TypeScript interfaces
■■■ constants.tsx           ← Seed data: outlets, menu, tables, staff
■■■ components/             ← React UI components (POS, KDS, Tables, Payments…)
■■■ services/
■   ■■■ db.ts               ← localStorage persistence layer
■   ■■■ geminiService.ts    ← Google Gemini AI (menu OCR + business insights)
■■■ package.json            ← Vite + React 19 + @google/genai
```

3. Project A — gusto_pos/ (Production Build)

Stack: Python FastAPI · SQLite · Next.js 16 · Tailwind v4

3.1 Folder Structure

```
gusto_pos/
■■■ backend/
■   ■■■ main.py          ← 4 API endpoints
■   ■■■ models.py        ← 5 database tables (SQLModel ORM)
■   ■■■ database.py      ← SQLite connection + session factory
■   ■■■ security.py      ← JWT auth, bcrypt hashing, Haversine geofencing
■   ■■■ requirements.txt
■   ■■■ .env             ← DB URL, SECRET_KEY, token expiry
■   ■■■ gusto_pos.db     ← SQLite file (auto-created on first run)
```

```
■
■■■ frontend/
■   ■■■ app/
■   ■   ■■■ page.tsx      ← Full POS dashboard: Login + 3 widgets
■   ■   ■■■ layout.tsx
■   ■■■ .env.local        ← NEXT_PUBLIC_API_URL=http://127.0.0.1:8000
■   ■■■ package.json      ← next 16, react 19, tailwind v4
■
■■■ launch.ps1            ← One-click: starts backend + frontend + opens browser
```

3.2 Database Schema (SQLite — 5 Tables)

Table	Key Columns	Purpose
customer	id, name, phone_number (UNIQUE)	Customer registration
otp_validation	id, phone_number, otp_code, expiry_time	OTP-based login
outlet	id, name, city, latitude, longitude, geofence_radius	Restaurant branches
menu_item	id, name, short_code (UNIQUE), base_price, is_veg, is_active	Searchable menu items
order	id, readable_id (INDEX), outlet_id (FK), table_number, status (0/1/2), total_amount, created_at	Orders

3.3 API Endpoints

Method	Path	Input	Output
POST	/auth/login	name, phone, otp	JWT access_token
GET	/pos/search?q=RD	search string	Array of menu items
POST	/order/validate-location	outlet_id, u_lat, u_lon	Geofence pass/fail + distance
POST	/order/create	outlet_id, table_no, amount	order_id, confirmation

3.4 Data Flow

```
User
 ■
 ■■ Login (OTP 123456) ■■■■■■■■■■■■■■■■■■■■■■■■ backend validates → JWT stored in browser
 ■
 ■■ Search "RD" ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ GET /pos/search?q=RD → SQLite menu_item
 ■                                                                    returns matches
 ■■ Get My Location ■■■■■■■■■■■■■■■■■■■■■■■■ browser geolocation API
 ■   { lat, lon } ■■■■■■■■■■■■■■■■■■■■■■■■■■■ POST /order/validate-location
 ■                                      Haversine formula, checks within 100m of outlet
 ■
 ■■ Create Order ■■■■■■■■■■■■■■■■■■■■■■■■■■■ POST /order/create → stored in SQLite orders
```

3.5 Security

- **JWT:** HS256, signed with SECRET_KEY, expires in 1440 min (24 h)
- **Password Hashing:** bcrypt with 12 rounds
- **Geofencing:** Haversine great-circle formula, default 100m radius per outlet
- **OTP:** Hardcoded '123456' for testing — replace with SMS provider in production

4. Project B — pos_1st_cut/gusto_pos/ (Advanced Build)

Stack: Python FastAPI (async) · PostgreSQL 15 · Redis · Alembic · Next.js 15 · WebSocket · Razorpay

4.1 Backend Folder Structure

```
backend/app/
■■■ main.py               ← FastAPI: CORS, WebSocket, route registration
■■■ core/
■   ■■■ database.py       ← Async SQLAlchemy + asyncpg (PostgreSQL)
■   ■■■ config.py         ← Environment config (pydantic-settings)
■   ■■■ security.py       ← JWT + bcrypt
■   ■■■ auth.py           ← Authentication service
■   ■■■ redis.py          ← Redis cache client
■   ■■■ ws_manager.py     ← WebSocket connection manager (broadcast by outlet)
■   ■■■ init_db.py        ← Schema init + seed data
■■■ models/
■   ■■■ base.py           ← Base class: UUID PK + created_at timestamp
■■■ modules/              ← One folder per feature domain
    ■■■ auth/             menu/   orders/   menu_items/   order_items/
    ■■■ organizations/    outlets/  users/    roles/
    ■■■ customers/        products/  inventory/  payments/
    ■■■ categories/       audit_logs/  sync_logs/   ws/
```

4.2 Module Pattern (every feature looks like this)

```
modules/orders/
■■■ controller.py   ← FastAPI route handlers  (@router.post, @router.get, …)
■■■ service.py      ← Business logic          (create_order, settle_table, …)
■■■ model.py        ← SQLAlchemy ORM class    (maps to DB table)
■■■ schema.py       ← Pydantic shapes         (request body, response body)
```

4.3 Database Schema (PostgreSQL — Hierarchical)

Level 1 — Root Entities

Table	Key Columns	Notes
organizations	id UUID PK, name, gst_number, created_at	Multi-tenant root
roles	id SERIAL PK, name UNIQUE, permissions JSONB	RBAC — permissions as JSON
customers	id UUID PK, name, phone_number UNIQUE	Customer profiles
otp_validations	id UUID PK, phone_number, otp_code, expiry_time	OTP login sessions

Level 2 — Outlets & Staff

Table	Key Columns	Notes
outlets	id UUID PK, org_id FK, location, lat, lon, geofence_radius	Restaurant branches
users	id UUID PK, username UNIQUE, hashed_password, outlet_id FK	Staff accounts
tables	id UUID PK, outlet_id FK, table_number, status	Physical tables per outlet

Level 3 — Menu

Table	Key Columns	Notes
menus	id UUID PK, name, outlet_id FK, is_active	One menu per outlet (or zone)
menu_categories	id UUID PK, menu_id FK, name	e.g. Appetizers, Main Course
menu_items	id UUID PK, category_id FK, name, price, image_url, is_veg, is_available	Individual dishes
item_modifiers	id UUID PK, menu_item_id FK, modifier_name, extra_price	Add-ons: extra cheese, etc.

Level 4 — Orders & Payments

Table	Key Columns	Notes
orders	id UUID PK, outlet_id FK, table_id FK, customer_id FK, status, total, created_at	Parent order record
order_items	id UUID PK, order_id FK, menu_item_id FK, quantity, price, modifiers JSON	Line items
payments	id UUID PK, order_id FK, amount, method, razorpay_id, status	Razorpay integration

Level 5 — Operations

Table	Key Columns	Notes
inventory	id UUID PK, outlet_id FK, product_id FK, stock_qty, reorder_level	Stock tracking per outlet
audit_logs	id UUID PK, entity_type, entity_id, action, user_id, created_at	Full activity trail
sync_logs	id UUID PK, device_id, last_sync_at, pending_orders	Offline device sync state

4.4 Frontend Structure (web_app/)

```
web_app/app/
■■■ page.tsx            ← Landing page
■■■ menu/page.tsx       ← Menu browsing (QR token decoded → outlet+table)
■■■ cart/page.tsx       ← Cart + checkout
■■■ login/page.tsx      ← Staff / customer login
■■■ pos/page.tsx        ← Manager dashboard
■■■ order/page.tsx      ← Order detail
■■■ checkout/page.tsx   ← Payment flow (Razorpay)
■■■ success/page.tsx    ← Order confirmation
■■■ tracking/page.tsx   ← Real-time order tracking via WebSocket

web_app/
■■■ contexts/           ← React global state (cart, auth, outlet)
■■■ hooks/              ← Custom React hooks
■■■ lib/
■   ■■■ api.ts          ← Axios API client (NEXT_PUBLIC_API_URL)
■   ■■■ cart-store.tsx  ← Cart state management
■■■ services/           ← Business logic helpers
■■■ types/              ← TypeScript interfaces
```

4.5 API Endpoints (pos_1st_cut)

Method	Path	Purpose
POST	/api/v1/menu/	Create a menu
GET	/api/v1/menu/{menu_id}	Get menu by ID
GET	/api/v1/menu/outlet/{outlet_id}	All menus for an outlet
POST	/api/v1/menu/items/	Create menu item

GET	/api/v1/menu/items/{item_id}	Get item by ID
GET	/api/v1/menu/items/category/{id}	Items by category
GET	/api/v1/orders/	List all orders
GET	/api/v1/orders/{order_id}	Order detail
POST	/api/v1/orders/takeaway	Create takeaway order
POST	/api/v1/orders/dinein	Create dine-in order
POST	/api/v1/orders/bill/{table_id}	Generate PDF bill
POST	/api/v1/orders/settle/{table_id}	Mark table as paid
WS	ws://.../ws/kitchen/{outlet_id}	Kitchen display real-time feed
WS	ws://.../ws/order/{order_id}	Customer order tracking feed

4.6 Complete Order Flow (End-to-End)

```
1. Customer scans QR code at table
   → Token decoded: outlet_id, table_id, zone (normal / AC)

2. Frontend: GET /api/v1/menu/outlet/{outlet_id}
   → PostgreSQL: query menus by outlet_id → return items

3. Customer browses and adds items to cart (React context, local state only)

4. Customer taps "Place Order"
   → POST /api/v1/orders/dinein { outlet_id, table_id, items[] }
   → Backend: INSERT into orders + order_items (PostgreSQL)
   → WebSocket: broadcast new order to kitchen/{outlet_id}

5. Kitchen Display System receives order via WebSocket
   → Staff marks "Preparing"
   → Backend: UPDATE orders SET status='preparing'
   → WebSocket: broadcast status to order/{order_id}

6. Customer /tracking page updates in real time (WebSocket)

7. Staff marks "Ready"
   → Customer tracking shows "Your order is ready"

8. Customer requests bill
   → POST /api/v1/orders/bill/{table_id}
   → Backend: generate PDF with reportlab, return download link

9. Customer pays via Razorpay (redirect/popup)
   → Razorpay webhook: POST /api/v1/payments
   → Backend: mark order as "paid", mark table as "available"
```

5. Root App — App.tsx (Offline-First Desktop POS)

Stack: Vite · React 19 · Tailwind v4 · localStorage · Google Gemini AI

5.1 UI Tabs

Tab	Component	Purpose
POS	POS.tsx	Place orders, assign to tables

Menu	MenuManagement.tsx	Add/edit items + Gemini OCR from photo
KDS	KDS.tsx	Kitchen Display System
Tables	Tables.tsx	Table status board
Payments	Payments.tsx	Settle bills
Analytics	Analytics.tsx + AdvancedAnalytics.tsx	Order trends and insights
Management	Management.tsx	Staff and table configuration

5.2 State Shape (persisted to localStorage)

```
AppState {
  outlets:         Outlet[]    // { id, name, location }
  currentOutletId: string      // Active outlet selection
  menu:            MenuItem[]  // { id, name, price, category, isVeg }
  orders:          Order[]     // { id, tableId, items[], total, status, timestamp }
  tables:          Table[]     // { id, name, status, capacity }
  staff:           Staff[]     // { id, name, role, phone, pin, isActive }
  isOnline:        boolean     // Network connectivity flag
  lastSynced:      number      // Epoch ms of last sync
}
Saved to localStorage["gusto_pos_state"] via services/db.ts
```

5.3 AI Features (Google Gemini)

- **Menu OCR:** Upload photo of physical menu → geminiService.ts sends base64 to gemini-3-flash-preview → returns structured JSON of items
- **Business Insights:** Order data passed to Gemini → LLM returns human-readable trend analysis

6. Other Projects

Project	Stack	Purpose
restaurant-pos-api/	Express 5 + SQLite	Bare-bones REST API — no frontend
restaurant-sim/	Express + JSON file	Demo simulator with HTML frontend
demo1/gusto_pos_fix/	FastAPI + Alembic	Fix iteration with proper DB migrations
socket/	Python + React	WebSocket experiments
Cricket_AI/	Python + Groq API	Unrelated: AI cricket commentary

7. Environment Variables

gusto_pos / backend .env

```
DATABASE_URL="sqlite:///./gusto_pos.db"
SECRET_KEY="75c27a7..."          # change in production
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=1440  # 24 hours
```

pos_1st_cut / backend .env

```
DATABASE_URL="postgresql+asyncpg://gusto_admin:gusto_password@localhost:5455/gusto_pos_v2"
SECRET_KEY="<openssl rand -hex 32>"
```

```
REDIS_URL="redis://localhost:6379/0"
ALGORITHM="HS256"
RAZORPAY_KEY_ID="..."
RAZORPAY_KEY_SECRET="..."
```

Frontend .env.local (all Next.js projects)

```
NEXT_PUBLIC_API_URL="http://127.0.0.1:8000"
GEMINI_API_KEY="..."              # root Vite app only
```

8. How to Start Each Project

Project	Command / Action	URL
gusto_pos (easiest)	Right-click launch.ps1 → Run with PowerShell	localhost:3000 + :8000/docs
pos_1st_cut backend	cd backend && uvicorn app.main:app --reload	localhost:8000
pos_1st_cut frontend	cd web_app && npm run dev	localhost:3000
Root Vite app	npm run dev (from Desktop/)	localhost:5173
restaurant-pos-api	npm run dev (from routes/)	localhost:3000

Test Credentials (all projects)

```
Phone:     any number
OTP:       123456
Admin PIN: 1234
Chef PIN:  0000
```

9. Project Maturity Map

Project	Maturity	Status
gusto_pos/	■■■■■■■■■■ 100%	Production-ready, tested, one-click launch
pos_1st_cut/	■■■■■■■■■■ 80%	Advanced features, active development
demo1/gusto_pos_fix/	■■■■■■■■■■ 60%	Fix iteration, migrations in place
restaurant-pos-api/	■■■■■■■■■■ 40%	API only, no frontend
restaurant-sim/	■■■■■■■■■■ 30%	Demo/sim only
Root App.tsx	■■■■■■■■■■ 60%	Offline POS + AI, no real backend
socket/ Cricket_AI/	■■■■■■■■■■ 20%	Experiments only

10. Technology Stack Summary

Layer	gusto_pos	pos_1st_cut	Root App
Backend Framework	FastAPI 0.104	FastAPI (async)	None
Database	SQLite	PostgreSQL 15	localStorage

ORM / DB Layer	SQLModel	SQLAlchemy 2.0 async + Alembic	—
Auth	JWT + bcrypt	JWT + bcrypt + RBAC	PIN
Real-time	—	WebSocket (ws_manager)	—
Cache	—	Redis	—
Payments	—	Razorpay	—
Frontend	Next.js 16	Next.js 15	Vite + React 19
Styling	Tailwind v4	Tailwind v3.4	Tailwind v4
AI / ML	—	—	Google Gemini
PDF Generation	—	reportlab	—